

Source Map Build Status  npm install source-map version 0.7.6 updated 6 months ago 234.5M downloads/week	This is a library to generate and consume the source map format described here. Use with Node \$ npm install source-map Use on the Web <script src="https://raw.githubusercontent.com/mozilla/source-map/master/dist/source-map.min.js" defer></script>	Table of Contents - Examples - Consuming a source map - Generating a source map - With SourceNode (high level API) - With SourceMapGenerator (low level API) - API - SourceMapConsumer - new SourceMapConsumer(rawSourceMap) - SourceMapConsumer.prototype.computeColumnSpans() - SourceMapConsumer.prototype.originalPositionFor()	- SourceMapConsumer.prototype.generatedPositionFor() - SourceMapConsumer.prototype.allGeneratedPositionsFor() - SourceMapConsumer.prototype.hasContentsOfAllSources() - SourceMapConsumer.prototype.sourceContentFor(source, returnNullOnMissing) - SourceMapConsumer.prototype.eachMapping(callback, context, order) - SourceMapGenerator - new SourceMapGenerator([startOfSourceMap]) - SourceMapGenerator.fromSourceMap(sourceMapConsumer) - SourceMapGenerator.prototype.addMapping(mapping)
1 8 <pre>var smc = new SourceMapConsumer(rawSourceMap); console.log(smc.sources); // ['http://example.com/www/js/one.js', // 'http://example.com/www/js/two.js'] // // CAAC,IAAI,IAAM,SAUA,GACIB,0AAOC,IAA</pre>	2 7 <pre>var rawSourceMap = { version: 3, file: 'min.js', names: ['bar', 'baz', 'n'], sources: ['one.js', 'two.js'], sourceRoot: 'http://example.com/www/' };</pre> Consuming a source map Examples	3 9 <pre>SourceNode.prototype.setSourceContent(sourceFileSourceContent) SourceNode.prototype.walk(fn) SourceNode.prototype.walkSourceContents(fn) SourceNode.prototype.join(sep) SourceNode.prototype.replaceRight(pattern, replacement) SourceNode.prototype.toString() SourceNode.prototype.toStringWithSourceMap({startOfSourceMap})</pre>	4 5 <pre>SourceMapGenerator.prototype.setSourceContent(sourceFileSourceContent) SourceMapGenerator.prototype.applySourceMap(sourceFileSourceMapPath) SourceMapGenerator.prototype.toString() SourceNode SourceNode.prototype.line, column, source, chunk, name SourceMapGenerator.prototype.toString() SourceMapConsumer.prototype.fromSourceMap(code, sourceMapConsumer, relativePath) SourceNode.prototype.prepend(chunk) SourceNode.prototype.add(chunk)</pre>
6 9 <pre>console.log(smc.generatedPositionFor({ line: 2, column: 10, name: 'n' })); console.log(smc.originalPositionFor({ line: 2, column: 28 })); // { source: 'http://example.com/www/js/two.js', // line: 2, // column: 10, // name: 'n' }</pre>	10 <pre>source: 'http://example.com/www/js/two.js', line: 2, column: 10 }); // { line: 2, column: 28 } smc.eachMapping(function (m) { // ... });</pre>	11 Generating a source map In depth guide: Compiling to JavaScript, and Debugging with Source Maps With SourceNode (high level API) function compile(ast) { switch (ast.type) { case 'BinaryExpression': return new SourceNode(ast.location.line,	12 <pre>ast.location.column, ast.location.source, [compile(ast.left), " + ", compile(ast.right)]); case 'Literal': return new SourceNode(ast.location.line, ast.location.column, ast.location.source, String(ast.value));</pre>
16 API Get a reference to the module: // Node.js var sourceMap = require('source-map'); // Browser builds // Browser builds	15 <pre>generated: { line: 10, column: 35 }, source: "foo.js", original: { line: 33, column: 2 }, name: "christopher" });</pre>	14 With SourceMapGenerator (low level API) // { code: '40 + 2', // map: [object] } SourceMapGenerator { }	13 <pre>// ... default: throw new Error("Bad AST"); } var ast = parse("40 + 2", "add.js"); console.log(compile(ast).toStringWithSourceMap({ file: 'add.js' }));</pre>

<pre>var sourceMap = window.sourceMap; // Inside Firefox const sourceMap = require("devtools/ toolkit/sourceemap/source- map.js");</pre> SourceMapConsumer A SourceMapConsumer instance represents a parsed source map which we can query for information about the original file positions by	giving it a file position in the generated source. new SourceMapConsumer(rawSourceMap) The only parameter is the raw source map (either as a string which can be JSON.parse'd, or an object). According to the	spec, source maps have the following attributes: - version: Which version of the source map spec this map is following. - sources: An array of URLs to the original source files. - names: An array of identifiers which can be referenced by individual mappings. - sourceRoot: Optional. The URL root from which all sources are relative. - sourcesContent: Optional. An array of contents of the original source files.	- mappings: A string of base64 VLQs which contain the actual mappings. - file: Optional. The generated filename this source map is associated with. <pre>var consumer = new sourceMap.SourceMapConsumer(rawSourceMap);</pre>
17	18	19	20
24	23	22	21
- line and column positions provided. The only argument is an object with the following properties: - line: The line number in the generated source. Line numbers in this library are 1-based (note that the underlying source map specification uses 0-based line numbers – this library handles the translation). - column: The column number in the generated source. Column numbers in this library are 0-based.	SourceMapConsumer.prototype.originalPositionFor <pre>// lastColumn: 9 }, // { line: 2, // column: 20, // lastColumn: 19 }, // { line: 2, // column: 10, // lastColumn: 19 }, // lastColumn: Infinity }]</pre> Returns the original source, line, and column information for the generated source's	<pre>consumer.computeColumnSpans(); // After: // { line: 2, // column: 20 }] // { line: 10, // column: 10 }, // { line: 2, // column: 20 }]</pre> SourceMapConsumer.prototype.computeColumn Compute the last column for each generated mapping. The last column is inclusive. <pre>consumer.allGeneratedPositionsFor({ line: 2, source: "foo.coffee" }) // [{ line: 2, // column: 1 }, // { line: 2, // column: 1 }]</pre>	SourceMapConsumer.prototype.computeColumn Compute the last column for each generated mapping. The last column is inclusive. <pre>consumer.allGeneratedPositionsFor({ line: 2, source: "foo.coffee" }) // [{ line: 2, // column: 1 }, // { line: 2, // column: 1 }]</pre>
bias: Either SourceMapConsumer.GREATEST_LOWER_BOUND or SourceMapConsumer.LEAST_UPPER_BOUND. Specifies whether to return the closest element that is smaller than or greater than the one we are searching for, respectively, if the exact element cannot be found. Defaults to SourceMapConsumer.GREATEST_LOWER_BOUND.	and an object is returned with the following properties: - source: The original source file, or null if this information is not available. - line: The line number in the original source, or null if this information is not available. The line number is 1-based. - column: The column number in the original source, or null if this information is not available. The column number is 0-based. - name: The original identifier, or null if this information is not available.	<pre>consumer.originalPositionFor({ line: 2, column: 10 }) // { source: 'foo.coffee', // line: 2, // column: 2, // name: null } consumer.originalPositionFor({ line: 999999999999999999, column: 999999999999999999 }) // { source: null, // line: null,</pre>	<pre>// column: null, // name: null }</pre> SourceMapConsumer.prototype.generatedPositionFor Returns the generated line and column information for the original source, line, and column positions provided. The only argument is an object with the following properties: source: The filename of the original source.
25	26	27	28
32	31	30	29
- column: Optional. The column number in the original source. The column number is 0-based. - line: The line number in the generated source, or null. The line number is 1-based. - column: The column number in the generated source, or null. The column number is 0-based.	either the line we are searching for or the next closest line that has any mappings. Otherwise, returns all mappings corresponding to the given line and either the closest column that has any offsets. The only argument is an object with the following properties: - source: The filename of the original source. - line: The line number in the original source. The line number is 1-based.	SourceMapConsumer.prototype.allGeneratedPositionsFor <pre>consumer.generatedPositionFor({ source: "example.js", line: 2, column: 10 }) // { line: 1, // column: 56 } Returns all generated line and column information for the original source, line, and column provided. If no column is provided, returns all mappings corresponding to a</pre>	- line: The line number in the original source. The line number is 1-based. - column: The column number in the original source. The column number is 0-based. - line: The line number in the generated source, or null. The line number is 1-based. - column: The column number in the generated source, or null. The column number is 0-based.

<pre>consumer.allGeneratedpositionsfor({ line: 2, source: "foo.coffee" }) // [{ line: 2, // column: 1 }, // { line: 2, // column: 10 }, // { line: 2, // column: 20 }]</pre>	SourceMapConsumer.prototype.hasContentsOfAllSources() Return true if we have the embedded source content for every source listed in the source map, false otherwise. In other words, if this method returns true, then consumer.sourceContentFor(s) will succeed for every source s in consumer.sources. <pre>// ... if (consumer.hasContentsOfAllSources()) {</pre>	<pre>consumerReadyCallback(consumer); } else { fetchSources(consumer, consumerReadyCallback); } // ...</pre>	SourceMapConsumer.prototype.sourceContentFor(source, optionalSourceMap) Returns the original source content for the source provided. The only argument is the URL of the original source file. If the source content for the given source is not found, then an error is thrown. Optionally, pass true as the second param to have null returned instead. <pre>consumer.sources // ["my-cool-lib.coffee"]</pre>
<pre>consumer.eachMapping(function (m) { console.log(m); }) // ... // { source: 'illmatic.js', // generatedLine: 1, // generatedColumn: 0, // originalLine: 1, // originalColumn: 0, // name: null } // { source: 'illmatic.js', // generatedLine: 1, // generatedColumn: 0, // originalLine: 1, // originalColumn: 0, // name: null }</pre>	<pre>SourceMapConsumer.prototype.eachMapping(context, order) // ... // null // in the source map", true); consumer.sourceContentFor("this is not original source/line/column and a generated line/column in this source map. callback: The function that is called with each mapping. Mappings have the form</pre>	<pre>consumer.sourceContentFor("this is not in the source map", true); // null // ... // "lib.coffee" consumer.sourceContentFor("my-cool- lib.coffee")</pre>	SourceMapGenerator.fromSourceMap(sourceMap) Creates a new SourceMapGenerator from an existing SourceMapConsumer instance. sourceMapConsumer The SourceMap. <pre>var generator = sourceMap.SourceMapGenerator.fromSourceMap(sourceMapConsumer);</pre>
<pre>// generatedLine: 2, // generatedColumn: 0, // originalLine: 2, // originalColumn: 0, // name: null } // ...</pre> SourceMapGenerator An instance of the SourceMapGenerator represents a source map which is being built incrementally.	new SourceMapGenerator([startOfSourceMap]) You may pass an object with the following properties: - file: The filename of the generated source that this source map is associated with. - sourceRoot: A root for all relative URLs in this source map. - skipValidation: Optional. When true, disables validation of mappings as they are added. This can improve performance but	should be used with discretion, as a last resort. Even then, one should avoid using this flag when running tests, if possible. <pre>var generator = new sourceMap.SourceMapGenerator({ file: "my-generated-javascript- file.js", sourceRoot: "http://example.com/app/ js/" });</pre>	SourceMapGenerator.prototype.addMapping(mapping) Add a single mapping from original source line and column to the generated source's line and column for this source map being created. The mapping object should have the following properties: - generated: An object with the generated line and column positions. - original: An object with the original line and column positions.
<pre>generator.setSourceContent("module- one.scm", fs.readFileSync("path/to/ module-one.scm")) SourceMapGenerator.prototype.applySourceMap(sourceFile, sourceMapPath))</pre> Applies a SourceMap for a source file to the SourceMap. Each mapping to the supplied source file is rewritten using the supplied	<pre>SourceMapGenerator.prototype.setSourceContent(sourceFile, sourceContent) Set the source content for an original source file. sourceFile: The URL of the original source file. sourceContent: The content of the source file.</pre>	<pre>source: The original source file (relative to the sourceRoot). name: An optional original token name for this mapping. generator.addMapping({ source: "module-one.scm", original: { line: 128, column: 0 }, generated: { line: 3, column: 456 } })</pre>	<pre>SourceMapGenerator.prototype.addMapping(mapping)</pre>

content and is passed the filename and source content. fn: The traversal function. <pre>var a = new SourceNode(1, 2, "a.js", "generated from a"); a.setSourceContent("a.js", "original a"); var b = new SourceNode(1, 2, "b.js", "generated from b"); b.setSourceContent("b.js", "original b");</pre> 65	<pre>var c = new SourceNode(1, 2, "c.js", "generated from c"); c.setSourceContent("c.js", "original c"); var node = new SourceNode(null, null, null, [a, b, c]); node.walkSourceContents(function (source, contents) { console.log("WALK:", source, ":", contents); }) // WALK: a.js : original a</pre> 66	<pre>// WALK: b.js : original b // WALK: c.js : original c</pre> SourceNode.prototype.join(sep) Like Array.prototype.join except for SourceNodes. Inserts the separator between each of this source node’s children. sep: The separator. 67	<pre>var lhs = new SourceNode(1, 2, "a.rs", "my_copy"); var operand = new SourceNode(3, 4, "a.rs", "="); var rhs = new SourceNode(5, 6, "a.rs", "orig.clone()"); var node = new SourceNode(null, null, null, [lhs, operand, rhs]); var joinedNode = node.join(" ");</pre> 68
SourceNode.prototype.toStringWithSourceMap() Returns the string representation of this tree of source nodes, plus a SourceMapGenerator which contains all the mappings between the generated and original sources. The arguments are the same as those to new SourceMapGenerator. <pre>var node = new SourceNode(1, 2, "a.js", ["a.js",] new SourceNode(3, 4, "b.js", "uno"), new SourceMapGenerator.</pre> 72	<pre>node.toString() // 'unodostresquatro'</pre> 71	<pre>node.replaceRight(/\s*\$/,""); Return the string representation of this source node. Walks over the tree and concatenates all the various snippets together to one string.</pre> SourceNode.prototype.toString() <pre>var node = new SourceNode(1, 2, "a.js", ["a.js",] new SourceNode(3, 4, "b.js", "uno"),</pre> 70	<pre>Call String.prototype.replace on the very right-most source snippet. Useful for trimming white space from the end of a source node, etc.</pre> - pattern: The pattern to replace. - replacement: The thing to replace the pattern with. SourceNode.prototype.replaceRight(pattern, replacement) 69
<pre>"dos", ["tres", new SourceNode(5, 6, "c.js", "quatro")]]()); node.toStringWithSourceMap({ file: "my-output-file.js" })</pre> 73	<pre>// { code: 'unodostresquatro', // map: [object // SourceMapGenerator] }</pre> 74	75	76
78	79	78	77